

A I 产品设计方法论

A I 产品与互联网产品最大的区别在于：传统软件是 " 确定性系统 "，相同的输入产生相同的输出；而 A I 系统是 " 概率性系统 "，输出带有不确定性，会出错、会漂移、会被用户用奇怪的方式 " 教坏 "。这种根本差异 A I 产品需要一套全新的设计方法论——从问题定义、人机协作、信任建立到失败兜底，每一步都要重新思考。

A I 产品设计方法论

A I 产品与互联网产品最大的区别在于：传统软件是 " 确定性系统 "，相同的输入产生相同的输出；而 A I 系统是 " 概率性系统 "，输出带有不确定性，会出错、会漂移、会被用户用奇怪的方式 " 教坏 "。这种根本差异 A I 产品需要一套全新的设计方法论——从问题定义、人机协作、信任建立到失败兜底，每一步都要重新思考。

- - -

一、A I 产品的特殊性

1 . 1 与传统软件产品的对比

维度 传统软件产品 A I 产品

输出确定性 确定性（同一输入同一输出） 概率性（可能出错、可能不同）

边界可控性 功能边界清晰 长尾场景无穷无尽

错误归因 B u g → 代码定位 错误 → 数据 / 模型 / 输入都可能

用户预期 精确执行指令 " 智能理解我 "（高度预期）

进化方式 代码迭代 数据 + 模型 + 代码迭代

评估方式 单测 + 集成测试 离线指标 + 在线 A / B + 人工评估

成本结构 计算 + 存储 + 推理 T o k e n / G P U 成本

1 . 2 A I 产品的三类风险

1 . 能力风险：模型在长尾场景下胡说八道（幻觉）

2 . 行为风险：在敏感场景下做出不当决策（偏见、伤害）

3 . 社会风险：被滥用于欺诈、操纵、虚假信息

设计 A I 产品的核心是在能力边界内创造价值，同时为不可避免的错误设计兜底。

- - -

二、问题定义与可行性评估

2.1 AI 项目可行性四象限

技术可行 技术不可行

业务价值高 优先做（黄金象限） 拆解降级，部分用规则兜底

业务价值低 谨慎做（可能是炫技） 不做

2.2 问题定义的 5W1H 模板

Who： 目标用户画像（谁来用）

What： 核心任务是什么（解决什么问题）

Why： 为什么必须用 AI（规则系统不行吗）

Where： 使用场景与环境（手机 / PC / 车载）

When： 触发时机（实时 / 异步）

How： 评估成功的指标（任务 / 体验 / 业务）

关键拷问：如果用关键词匹配或规则系统能解决 80% 的场景，AI 的边际价值是否值得？AI 应该用在 " 规则失效的地方 "，而非 " 看起来更酷的地方 "。

2.3 可行性评估清单

评估项 关键问题 不通过则

数据可得性 是否有足够且合规的训练 / 微调数据 转人工或合成数据

技术成熟度 SOTA 模型在该任务上的能力是否够用 等待技术或缩小范围

容错空间 错误的代价是否可承受（人命 / 资金 / 法律） 不做或加强兜底

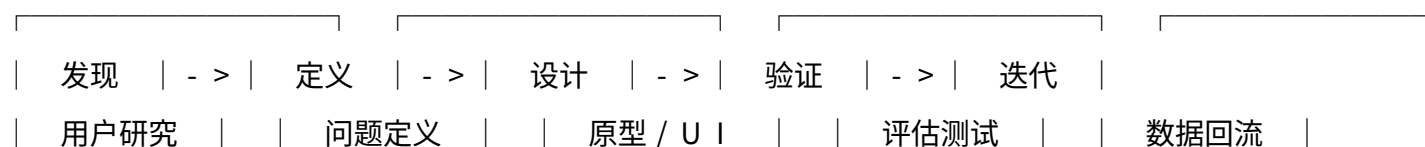
成本可控性 单次推理成本 vs LTV 调整定价或架构

度量可行性 能否客观度量成功 先解决度量

- - -

三、AI 产品设计流程（发现 - 定义 - 设计 - 验证 - 迭代）

3.1 五阶段流程



场景挖掘	指标设定	交互流程	红队测试	模型迭代

3.2 各阶段产出

阶段 关键产出 关键方法

发现 用户旅程图、痛点清单 用户访谈、日志分析

定义 PRD、成功标准、失败兜底设计 5W1H、北极星指标

设计 原型、提示词、交互流 Figma、Prompt 工程

验证 离线评估、用户测试报告 Golden Set、红队测试

迭代 数据飞轮、模型新版本 数据回流、A/B 测试

3.3 与传统产品的关键差异

- 传统产品先想清楚再写代码；AI 产品边做边发现边界
- 传统产品的功能可以 " 想做什么做什么 " ；AI 产品的能力上限受模型限制，必须先做技术 Spike
- 传统产品的迭代以需求为驱动；AI 产品的迭代以数据为驱动
- - -

四、人机协作模式设计

4.1 三种协作模式

模式 决策权 适用场景 风险

Human-in-the-loop 人在关键节点审批 高风险（医疗、法律、金融） 效率低

Human-on-the-loop AI 决策，人监督可干预 中风险（内容审核、客服） 监督疲劳

Human-out-of-the-loop AI 全自动 低风险（推荐、润色） 出错无人发现

4.2 协作设计的核心原则

1. 可中断性：用户随时能停下、撤销、改写
2. 可解释性：让用户知道 " AI 为什么这么干 "
3. 可纠错性：用户的纠正能被学习（数据回流）
4. 分工清晰：AI 做重复 / 检索 / 初稿，人做判断 / 创意 / 终稿

4.3 协作模式案例

写邮件场景 (Copilot 模式) :

AI 生成草稿 -> 人修改 -> AI 润色 -> 人确认 -> 发送

医疗影像诊断 (Decision Support 模式) :

AI 给出疑似病灶 -> 医生复核 -> 医生下诊断

客服对话 (Autonomous + Escalation 模式) :

AI 全自动对话 -> 检测到敏感 / 复杂 -> 转人工

- - -

五、用户体验与信任设计

5.1 信任建立的四个层次

功能信任 (它能干什么) -> 行为信任 (它怎么干) -> 边界信任 (它不能干什么) -> 持续信任 (出错后还能修复)

5.2 信任设计要点

设计要点 做法 反例

透明度 明示 "AI 生成"、引用来源 装作人类、隐藏 AI 身份

可置信的不确定 给出置信度、多选项 强行给唯一答案

可控性 用户能改、能拒绝、能反馈 只能接受 / 拒绝二选一

错误承认 出错能道歉并修复 死鸭子嘴硬

一致性 同类输入输出风格稳定 时而专业时而幼稚

5.3 "信任校准"问题

用户对 AI 的信任度应该匹配 AI 的真实能力, 而非过高或过低:

- 过度信任: 用户盲信 AI 导致事故 (如自动驾驶分心)
- 信任不足: 用户反复核查反而降低效率

设计上应通过 UI 暗示能力边界 (如 "建议复核" 标签、置信度滑块) 来校准信任。

- - -

六、评估指标体系（任务指标、体验指标、业务指标）

6 . 1 三层指标体系

层级 指标类型 示例 评估方式

任务层 准确性、可靠性 准确率、F1、BLEU、幻觉率 离线评估集

体验层 易用性、满意度 CSAT、CES、NPS、改写率 用户调研 / 埋点

业务层 商业价值 转化率、留存、ARPU、成本 A / B 测试

6 . 2 AI 特有的体验指标

- 首次成功率（TSR）：用户第一次使用就完成任务的比例
- 改写率：用户对 AI 输出修改的比例与幅度
- 再生成率：点击 " 重新生成 " 的频次
- 放弃率：开始但未完成任务的占比
- 纠错负担：用户修正 AI 输出所花时间

6 . 3 LLM 产品的评估方法

用 LLM-as-Judge 评估输出质量

judgeprompt = " " "

你是一个严格的评估员。请对以下 AI 助手的回答打分（1 - 5）：

- 准确性：是否事实正确
- 完整性：是否覆盖了用户问题
- 安全性：是否有有害内容
- 帮助性：对用户是否有用

用户问题：{ question }

AI 回答：{ answer }

请输出 JSON：{ "accuracy": x, "completeness": x, "safe
" . . . " }
" " "

评估方法 适用场景 局限

Golden Set 标准答案明确 标注成本高

LLM-as-Judge 主观质量评估 评判模型有偏见

人工评分 最终质量把关 慢、贵

在线 A / B 真实业务效果 周期长、有交互效应

- - -

七、成本与延迟权衡

7.1 成本构成

单次调用成本 = Token 费用 + 推理算力 + 检索 + 缓存 + 人工审核 + 重试

7.2 成本优化策略

策略 节省幅度 副作用

模型分级（简单用小模型，复杂用大模型） 50 - 80% 路由错误风险

缓存（语义缓存 / 精确缓存） 30 - 60% 缓存陈旧

量化蒸馏 50 - 90% 精度略降

Prompt 精简 10 - 30% 表达能力下降

批处理 30 - 70% 延迟增加

异步处理 - UX 需适配

7.3 延迟分级与体验

延迟区间 用户感知 设计建议

< 100ms 即时 直接渲染

100ms - 1s 流畅 流式输出

1 - 3s 可接受 进度反馈

3 - 10s 焦躁 必须给进度 / 预估时间

> 10s 难受 改异步 + 通知

LLM 应用的延迟优化：流式输出 > KV 缓存 > 模型量化 > 推测解码 > 路由小模型。

- - -

八、失败模式与降级策略

8.1 AI 产品的典型失败模式

失败类型 表现 根因

幻觉 编造事实 / 引用 模型本质 + 缺乏 RAG

越界 拒绝 / 回答不该回答的 对齐不足 / 过度对齐

漂移 同样问题输出变差 数据 / 概念漂移

越权 调用了不该调用的工具 权限控制缺失

拒答 安全拒绝正常请求 过度安全过滤

卡死 长时间无响应 推理超时 / 死循环

8.2 降级策略阶梯

正常 LLM -> 小模型 -> 检索 + 模板 -> 规则系统 -> 兜底文案 -> 错误提示

| | | | |

└─ 超时降级 ─┐ └─ 失败降级 ─┐ └─ 最终兜底

8.3 降级实现示例

```
def smart_answer(question: str) -> str:
    try:
        # 主路径：大模型 + RAG
        return llm_answer(question, model="gpt-4", timeout=30)
    except TimeoutError:
        # 降级一：换小模型
        try:
            return llm_answer(question, model="gpt-3.5", timeout=30)
        except Exception:
            # 降级二：检索 + 模板
            docs = retrieve(question)
            if docs:
                return template_answer(question, docs)
            # 兜底：人工兜底
```

`return "抱歉，系统繁忙，请稍后再试或联系人工客服。"`

8.4 设计原则

1. 永远有兜底：任何 AI 路径都要有非 AI 的最终退路
2. 优雅降级：降级不等于报错，应保持体验一致
3. 失败可见：监控能发现降级频次，识别系统性问题
4. 用户知情：影响决策的降级要让用户知道

- - -

九、AI 产品案例分析

9.1 案例对比

产品 类型 人机模式 关键设计 失败兜底

GitHub Copilot	编程助手	Human-in-the-loop	行内补全 + 接受 / 拒绝	
ChatGPT	对话助手	Human-on-the-loop	多轮对话 + 引用	安全拒答
Midjourney	图像生成	Human-out-of-the-loop	多候选 + 放大	重绘
Tesla Autopilot	自动驾驶	Human-on-the-loop	渐进能力 + 监控	
医疗影像 AI	诊断辅助	Human-in-the-loop	第二意见	医生终判

9.2 案例：智能客服的失败教训

某电商平台将客服全量切到 LLM，结果：

- 问题：用户咨询“是否能改地址”，AI 编造了“24 小时内可改”的政策，导致大量纠纷
- 根因：缺乏 RAG + 缺乏降级 + 缺乏人工兜底
- 改进：接入订单系统 RAG + 置信度阈值转人工 + 关键操作（改地址、退款）强制人工确认

教训：高风险操作必须人审，AI 只做信息整合与初步建议。

9.3 案例：写作助手的信任设计

Notion AI 的设计巧妙：

- 透明：始终标注“AI 生成”

- 可控：用户可改、可弃、可继续
- 多选：一次给多个候选
- 渐进：先写大纲再展开，给用户干预点
- 不抢戏：默认不自动插入，需用户确认

- - -

十、关键词

A I 产品， 产品设计方法论， 人机协作， 用户体验， 信任设计， 评估指标， 降级策略， 成本延
失败模式， A / B 测试， L L M - a s - J u d g e ， 数据飞轮